1. Edges: $A \to B(5)$, $A \to C(3)$, $A \to D(2)$, $B \to C(-10)$, $C \to D(-2)$; source A.

iter	**extract**	$\boldsymbol{d(A)}$	$\boldsymbol{d(B)}$	$\boldsymbol{d(C)}$	$\boldsymbol{d(D)}$
0	—	0	∞	∞	∞
1	$A(0)$	0	5	3	2
2	$D(2)$	0	5	3	2
3	$C(3)$	0	5	3	1 (updated after extraction)
4	$B(5)$	0	5	-5 (updated after extraction)	1

Q: $\{A, B, C, D\} \to \{B, C, D\} \to \{B, C\} \to \{B\} \to \varnothing$.

Problem: Dijkstra finalizes D at 2 and C at 3, but later relaxations improve them after extraction ($C \to D$: $3 - 2 = 1 < 2$; $B \to C$: $5 - 10 = -5 < 3$). With negative edges the extracted-minimum is no longer final. The true distances are $d(C) = -5$ and $d(D) = d(C) - 2 = -7$; Dijkstra's greedy assumption (that a closest unfinalized vertex needs no further improvement) fails because a longer prefix can become cheaper via a negative edge.

2. Modified Dijkstra (re-push on every successful relax). Distances after each extraction:

extract	$\boldsymbol{d(B)}$	$\boldsymbol{d(C)}$	$\boldsymbol{d(D)}$	$\boldsymbol{d(E)}$	$\boldsymbol{d(F)}$	$\boldsymbol{d(G)}$	**pushed**
$A(0)$	4	17	∞	∞	∞	∞	—
$B(4)$	4	17	8	∞	∞	∞	—
$D(8)$	4	17	8	10	16	∞	—
$E(10)$	4	17	8	10	16	12	—
$G(12)$	4	17	8	10	16	12	—
$C(17)$	4	17	4	10	16	12	D
$D(4)$	4	17	4	6	12	12	E, F
$E(6)$	4	17	4	6	12	8	G
$G(8)$	4	17	4	6	12	8	—
$F(12)$	4	17	4	6	12	6	G
$G(6)$	4	17	4	6	12	6	—
$F(12)$	4	17	4	6	12	6	—

(a) A, B, D, E, G, C, D.

(b) E, G, F, G, F — the queue is then empty (12 extractions in total; the last one, $F(12)$, is F's remaining duplicate entry from the initial queue).

Final: $d = (A : 0, B : 4, C : 17, D : 4, E : 6, F : 12, G : 6)$.

AB,
BD,
AC,
CD,
DE,
EG,
DF,

3. Bellman-Ford, edge order FG .

edge	AB	BD	AC	CD	DE	EG	DF

iter 1	$B:4$	$D:8$	$C:17$	$D:4$	$E:6$	$G:8$	$F:12$

then FG: $G: 12-6=6<8 \to G:6$. After iteration 1: $(A,B,C,D,E,F,G) = (0,4,17,4,6,12,6)$.

Iteration 2: no edge relaxes (all tentative distances already optimal) $\Rightarrow$ no change; the algorithm may stop.

Shortest-path tree (via π): $A \to B$, $A \to C$, $C \to D$ $(17-13)$, $D \to E$, $D \to F$, $F \to G$ $(12-6)$; distances $(0,4,17,4,6,12,6)$.

4.

(a) By independence, a route is open with probability $\prod_{e\in \text{ route}}(1-p(e))$, hence closed with probability $1-\prod_e(1-p(e))$. Minimizing the probability of being closed is the same as **maximizing** $\prod_e(1-p(e))$. Take logs: $\log\prod_e(1-p(e)) = \sum_e \ln(1-p(e))$, and ln is increasing, so this is the route **minimizing** $\sum_e(-\ln(1-p(e)))$. Define weights $w(e) = -\ln(1-p(e)) \geq 0$ (since $p(e)\leq 1$) and run Dijkstra with these nonnegative weights; the shortest path is the route with minimum probability of being closed. (Edge case: $p(e)=1$ gives $w=\infty$ — the road is surely closed, so any route through it has closed-probability 1; such edges are effectively unusable.)

(b) To **maximize** a product of edge weights $q(e)\in[0,1]$: run Dijkstra with (i) the tentative value $d(v)$ meaning "best (largest) product so far", initialized $d(s)=1$, others 0; (ii) relaxation replaced by $d(v) < d(u)\cdot q(u,v) \Longrightarrow d(v) = d(u)\cdot q(u,v)$; (iii) EXTRACT-MAX instead of EXTRACT-MIN. Correctness: prefix optimality holds for any nonnegative weights (multiplication by a nonnegative number preserves order, so a better prefix always gives a better whole path). What needs $q\leq 1$ is the extraction step: the remaining suffix of any path has product ≤ 1, so no later relaxation can raise a value above the current maximum — the extracted vertex is final, exactly as nonnegative weights guarantee in Dijkstra. (With weights $q>1$ the greedy fails: e.g. $s\to a(0.1), s\to y(0.9), a\to y(100)$ extracts y first at 0.9 though the optimum is 10.) For our problem set $q(e)=1-p(e)$ (probability the road is open); the max-product path is the route maximizing the probability of being open, i.e., minimizing the probability of being closed.

5. Dijkstra started at s never fails on such a graph (assume there is no negative cycle reachable from s; otherwise shortest distances are not defined and the question is void).

Claim: every vertex is extracted with $d(u)=\delta(u)$. Suppose not, and let y be the **first** wrongly extracted vertex. Take a shortest path $s=v_0\to v_1\to\ldots\to v_k=y$, and let v_i be the last vertex on the path extracted before y ($v_0=s$ qualifies, since s is extracted first). Then v_{i+1} is still in Q when y is extracted, so EXTRACT-MIN gives

$$d(y)\leq d(v_{i+1}).$$

Bound on $d(v_{i+1})$: when v_i was extracted, relaxing (v_i,v_{i+1}) set $d(v_{i+1})\leq d(v_i)+w(v_i,v_{i+1})$, and d only decreases afterwards.

- If $i=0$: $d(v_0)=\delta(s)=0$, so $d(v_1)\leq w(s,v_1)=\delta(s,v_1)$ (the one-edge prefix of a shortest path is shortest — this is the only place a negative edge, namely one leaving s, may appear, and it causes no harm).
- If $i\geq 1$: v_i was extracted before y, so by minimality of y it was extracted correctly: $d(v_i)=\delta(v_i)$; hence $d(v_{i+1})\leq\delta(v_i)+w(v_i,v_{i+1})=\delta(v_{i+1})$.

In both cases $d(v_{i+1}) \le \delta(v_{i+1})$. Moreover the suffix $v_{i+1} \to ... \to y$ of the shortest path contains no edge leaving s (its tails are $v_{i+1}, ..., v_{k-1} \neq s$), so all its edges are positive and

$$\delta(v_{i+1}) \le \delta(v_{i+1}) + \underbrace{w(v_{i+1}, v_{i+2}) + ... + w(v_{k-1}, y)}_{\text{suffix weight}} = \delta(y).$$

Combining: $d(y) \le d(v_{i+1}) \le \delta(v_{i+1}) \le \delta(y)$, contradicting $d(y) > \delta(y)$. So no wrongly extracted vertex exists, and Dijkstra is correct. (If $i + 1 = k$, i.e., y itself was relaxed by v_i, the same chain reads $d(y) \le \delta(y)$ directly.) □

6. Run Bellman-Ford from u but with only k passes over all edges (instead of $|V| - 1$): after pass i, $d(v)$ equals the weight of the shortest path from u to v using at most i edges (standard induction on the DP). Since every shortest path has at most k edges, after k passes $d(v) = \delta(u, v)$. The path itself is recovered from the π pointers maintained by RELAX, at no extra cost. Time: k passes $\times\, O(|E|)$ relaxation $= O(k\ |E|)$.

7. For a landmark ℓ, precompute (by two Dijkstra runs on G and on the reverse graph) $d(x, \ell)$ and $d(\ell, x)$ for all x. For target t, the potential (lower bound on $d(v, t)$) of landmark ℓ is

$$\varphi_\ell(v) = \max\{d(v, \ell) - d(t, \ell), \quad d(\ell, t) - d(\ell, v)\}.$$

Both terms are lower bounds on $d(v, t)$ by the triangle inequality ($d(v, t) \ge d(v, \ell) - d(t, \ell)$ and $d(v, t) \ge d(\ell, t) - d(\ell, v)$), and both vanish at $v = t$. (The terms must be finite; a landmark with an infinite distance to/from t or v is dropped or its term clamped — consistency below only uses finite values.)

Consistency (single landmark). Need: $\varphi(u) \le w(u, v) + \varphi(v)$ for every edge (u, v), and $\varphi(t) = 0$. The second is clear. For the first term, using the triangle inequality $d(u, \ell) \le w(u, v) + d(v, \ell)$:

$$d(u, \ell) - d(t, \ell) \le w(u, v) + (d(v, \ell) - d(t, \ell)).$$

For the second term, $d(\ell, v) \le d(\ell, u) + w(u, v)$ gives $d(\ell, u) \ge d(\ell, v) - w(u, v)$, so

$$d(\ell, t) - d(\ell, u) \le d(\ell, t) - d(\ell, v) + w(u, v) = w(u, v) + (d(\ell, t) - d(\ell, v)).$$

A max of two quantities each $\le w(u, v) + \varphi_\ell(v)$ is itself $\le w(u, v) + \varphi_\ell(v)$. So φ_ℓ is consistent. □

Multiple landmarks. The combined potential is $\varphi(v) = \max_\ell \varphi_\ell(v)$. A maximum of consistent potentials is consistent:

$$\varphi(u) = \max_\ell \varphi_\ell(u) \le \max_\ell (w(u, v) + \varphi_\ell(v)) \le w(u, v) + \max_\ell \varphi_\ell(v) = w(u, v) + \varphi(v),$$

and $\varphi(t) = \max_\ell \varphi_\ell(t) = \max_\ell 0 = 0$. Hence landmark potentials are always consistent (and admissible, being lower bounds on $d(v, t)$). □

8. (DPV 4.7) Algorithm:

Step 1. Compute $d_T(s, v)$ for all v in $O(V + E)$: propagate along the tree from s (DFS/BFS on T: $d_T(s, \text{child}) = d_T(s, \text{parent}) + w$). (This also checks T is really a tree rooted at s reaching all of V.)

Step 2. For **every** edge $(u, v) \in E$, check $w(u, v) + d_T(s, u) \ge d_T(s, v)$. Tree edges pass with equality by Step 1. Accept iff every edge passes. Total time $O(V + E)$: linear.

Correctness.

($\Rightarrow$) If T is a shortest-path tree, $d_T(s,x) = \delta(s,x)$ for all x, and for any edge (u,v), $\delta(s,v) \leq \delta(s,u) + w(u,v)$ (subpaths of shortest paths / a path to u extended by one edge reaches v) — exactly the test. All edges pass.

($\leq$) Suppose all edges pass but T is not a shortest-path tree: some v has $\delta(s,v) < d_T(s,v)$ (note $d_T(s,v) \geq \delta(s,v)$ always, since d_T is a real path length). Take any shortest path $s = v_0 \to v_1 \to ... \to v_k = v$. Summing the passing tests along its edges,

$$d_T(s,v_k) \leq d_T(s,v_0) + \sum_{i=1}^{k} w(v_{i-1},v_i) = 0 + \delta(s,v) < d_T(s,v_k),$$

a contradiction. (Same telescoping around any reachable negative cycle gives $d_{T(s,v)} < d_{T(s,v)}$, so the test also rejects graphs where no shortest-path tree exists.) Hence $d_T(s,v) = \delta(s,v)$ for all v, i.e., T is a shortest-path tree. $\square$